The Constitutional Codex

Three-Layer Agent-Executable Specification for Complete System Realization

But implementation correctness is only one layer of a complete system specification.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

The Implementation Codex (CX-I) demonstrated that software construction can be made deterministic through phase-gated, AI-executable specification. But implementation correctness is only one layer of a complete system specification. The Implementation Codex tells an agent *how to build* a system – it does not explain *why this configuration was chosen*, what alternatives exist, or what would need to change if requirements shifted.

This paper introduces the **Constitutional Codex** (CX:AES): a three-layer specification framework that extends the Implementation Codex with two additional layers – the Constitutional Specification (CX-S), which defines the field of possibility and domain constraints, and the Constitutional Configuration (CX-C), which documents the selected design, its rationale, and the topology of alternative configurations.

Together, the three layers form a complete, corrigible, agent-executable specification: the CX-S declares what *could* be built, the CX-C declares what *was chosen* and why, and the CX-I declares *how to build it*. All three layers are updateable through observation and insight, using the same journal-driven refinement cycle that proved effective for the Implementation Codex.

The central claim is: *a system is fully specified only when its possibility space, configuration rationale, and construction plan are all explicit, all corrigible, and all agent-executable.*

---

1. Introduction

The Implementation Codex solved a specific problem: given a settled architecture, how do you specify it precisely enough that an AI agent can build it correctly on first pass? The Implementation Codex source reports eight builds across

two models and two languages as evidence for phase-gated construction with journal-driven refinement. This paper does not independently reproduce that evidence; it treats the reported result as a motivating predecessor claim.

But the Implementation Codex is deliberately silent on a deeper question: *why was this architecture chosen?*

When the Implementation Codex encounters a genuine design ambiguity – receipts could be append-only or versioned, the observer actor could be empty string or a named system identity, hash deserialization could use approach A or approach B – it escalates to the Governor. The Governor makes a decision. The decision is recorded in the version history. But the *reasoning space* – why this choice was correct, what other choices were possible, what conditions would make a different choice better – is not captured.

This means three things are lost:

First, the design rationale. A new Governor inheriting the Codex sees *what* was decided but not *why*. The decision appears arbitrary because the alternatives and their tradeoffs are not documented.

Second, the configuration topology. The Codex specifies one point in a space of valid configurations. It does not map the space. An agent encountering a new requirement cannot determine whether the requirement is satisfiable within the current configuration, requires a reconfiguration, or is inadmissible under fundamental domain constraints.

Third, the corrigibility structure. The Implementation Codex is updateable through observation – but only at the implementation level. Observations that challenge the *architecture itself* (not just its specification) have no formal mechanism for propagation. They become Governor judgment calls rather than systematic specification refinement.

The Constitutional Codex addresses these gaps by adding two layers above the Implementation Codex.

---

2. The SCI Framework

The Constitutional Codex is structured according to the Specification-Configuration-Implementation (SCI) framework, which treats every engineered system as existing at three distinct levels.

2.1 Specification (CX-S)

The specification layer defines the **field of possibility**. It answers: what are the fundamental constraints of this domain? What could be built? What is admissible and what is inadmissible regardless of configuration?

For a governance system, CX-S might state: "Every mutation must have a traceable cause. This is a domain invariant – no valid governance system omits it. The cause may be a receipt, a log entry, an audit record, or any other mechanism that provides traceability. The choice of mechanism is a configuration decision, not a specification constraint."

CX-S distinguishes between:

Domain invariants – properties that hold in every valid configuration of this system type. These are the physics of the domain. Violating them produces a system that is not a member of the class.

Configuration parameters – dimensions along which valid configurations vary. Each parameter has a range of admissible values. The CX-C selects a specific value.

Inadmissible regions – combinations of parameter values that violate domain invariants. These are the configurations that *cannot* work, regardless of how well they are implemented.

2.2 Configuration (CX-C)

The configuration layer documents the **selected design**. It answers: within the field of possibility, which specific configuration was chosen and why?

For a governance system, CX-C might state: "Traceability is achieved via append-only receipts (not log entries or audit records). This was chosen because append-only receipts provide independent verifiability – any party can recompute the hash and confirm the receipt chain without trusting the system that produced it. Log entries were rejected because they are mutable. Audit records were rejected because they require a trusted auditor."

CX-C documents:

Selected configuration – the specific parameter values chosen from the CX-S possibility space.

Rationale – why each selection was made, referencing the tradeoffs involved.

Alternatives considered – what other configurations were evaluated, and why they were rejected or deferred.

Reconfiguration triggers – what observations or requirement changes would cause this configuration to be reconsidered. These are the conditions under which the CC should be reopened.

Configuration topology – the map of optimal, suboptimal, and inadmissible configurations in the possibility space. Not every configuration needs to be enumerated, but the topology should be described well enough that an agent can classify a new configuration as admissible or inadmissible.

2.3 Implementation (CX-I)

The implementation layer is the **build plan**. It answers: given this specific configuration, how do you construct the system deterministically?

This is the Implementation Codex as previously described: dependency graph, phase linearization, phase specifications with exact types and pseudocode, patterns library, validation protocol with gate tests, and language binding guides. The CX-I is the layer that agents execute directly.

The key property of the CX-I is that it treats the architecture as settled. The CX-I does not explain *why* receipts are append-only – it specifies *how* to implement append-only receipts with SQL triggers. The explanation belongs in the CX-C. The domain constraint that makes traceability necessary belongs in the CX-S.

---

3. Corrigibility Across Layers

A constitutional system is one that specifies how it can be changed. The Constitutional Codex applies this principle to itself: all three layers are corrigible through a governed process.

3.1 Implementation Corrigibility

The CX-I is corrected through the journal-driven refinement cycle demonstrated in the Implementation Codex paper. Build observations are classified (GAP, FRICTION, AMBIGUITY), integrated into the specification, and validated through cross-model builds. This cycle operates within a fixed configuration – the architecture doesn't change, only its specification.

3.2 Configuration Corrigibility

The CX-C is corrected when implementation experience reveals that the selected configuration is suboptimal or inadequate. An observation at the CX-I level – "this invariant is structurally harder to enforce than expected" – may propagate upward to the CX-C level: "should we use a different enforcement mechanism?"

Configuration changes are more consequential than implementation changes. A CX-I change (adding a deserialization mechanism to a phase spec) affects one build step. A CX-C change (switching from append-only receipts to versioned receipts) may invalidate multiple phases, patterns, and gate tests. The CX-C should document the blast radius of each configuration choice – which parts of the CX-I would need to change if this configuration parameter were modified.

3.3 Specification Corrigibility

The CX-S is corrected when new theoretical insights change the field of possibility. A domain invariant previously thought fundamental may be revealed as a

configuration choice. A previously admissible region of the configuration space may be shown to be inadmissible under a newly discovered constraint.

Specification changes are the most consequential. A CX-S change may invalidate the current configuration (CX-C), which invalidates the current implementation (CX-I). The CX-S should document which domain invariants are empirically validated versus theoretically assumed.

3.4 Upward and Downward Propagation

Observations propagate upward: a build failure (CX-I) may reveal a configuration limitation (CX-C), which may reveal a specification gap (CX-S).

Constraints propagate downward: a new domain invariant (CX-S) constrains the admissible configurations (CX-C), which constrains the implementation (CX-I).

The journal mechanism extends across all three layers. An observation recorded during a build may be classified as:

- **CX-I observation**: specification gap at the implementation level (fix the phase spec)
- **CX-C observation**: the selected configuration doesn't handle this case (consider reconfiguration)
- **CX-S observation**: the domain has a constraint we didn't account for (update the possibility space)

Each classification determines where the correction is applied and how far its effects propagate.

---

4. The Configuration Topology

One of the CX-C's most important functions is mapping the configuration topology – the space of possible configurations and their properties.

4.1 Optimal Configurations

A configuration is optimal when it satisfies all domain invariants (CX-S) and minimizes tradeoffs for the stated requirements. There may be multiple optimal configurations for different requirement sets.

4.2 Suboptimal Configurations

A configuration is suboptimal when it satisfies all domain invariants but accepts unnecessary tradeoffs. Suboptimal configurations are valid – they work – but a better configuration exists for the same requirements.

4.3 Inadmissible Configurations

A configuration is inadmissible when it violates a domain invariant. No implementation of an inadmissible configuration can produce a correct system. The CX-S defines the boundary between admissible and inadmissible; the CX-C maps specific configurations relative to that boundary.

4.4 Why Topology Matters

Without topology, every configuration decision is a point decision – "we chose A, not B." With topology, configuration decisions become navigational – "we are at point P in the space, alternatives Q and R are nearby, the inadmissible region is in direction D, and moving toward S would improve property X at the cost of property Y."

This enables agents to reason about configuration changes without Governor intervention for simple cases. An agent encountering a new requirement can consult the CX-C topology: if the requirement is satisfiable within the current configuration's neighborhood, the agent can propose a specific modification. If the requirement requires crossing an inadmissible boundary, the agent knows to escalate.

---

5. Relationship to the Implementation Codex

The Constitutional Codex does not replace the Implementation Codex. It contextualizes it.

The CX-I remains the artifact that agents execute directly. Its structure (six document classes, phase-gated construction, gate tests, journal-driven refinement) is unchanged. The CX-S and CX-C are authored by the Governor and consulted by agents when they encounter design-level questions that the CX-I cannot answer.

The practical workflow:

System design – the Governor, aided by governor-coder skill sessions, develops the CX-S (domain analysis) and CX-C (configuration selection). This produces the architectural decisions that the CX-I will specify.

Implementation specification – the Governor, using the codex-author skill, produces the CX-I from the CX-C. The CX-I specifies the selected configuration with enough precision for deterministic construction.

Construction – an agent, using the codex-governed-session skill, builds from the CX-I. The CX-S and CX-C are available for reference but the agent builds from the CX-I.

Refinement – build observations are classified by layer (CX-I, CX-C, or CX-S) and corrections are applied at the appropriate level. Most observations are

CX-I level. Configuration and specification observations are rarer but more consequential.

---

6. Empirical Grounding

The Constitutional Codex framework is grounded in the empirical results of the Implementation Codex, which demonstrated the CX-I layer across eight builds.

Several observations from those builds were, in retrospect, CX-C level observations rather than CX-I level:

The C4 observer exemption. The Phase 1 validator rule "authority_present must equal (actor is non-empty)" conflicted with the Phase 3 observer convention "actor is 'observer:system' but authority_present is false." This is a configuration decision – observers are a structurally different category from governed actors. The CX-I recorded it as a validator fix. The CX-C would record it as a design decision: "Observer receipts name a system identity for traceability without claiming governance authority. The validation rule has an observer exemption because observers occupy a different position in the authority model than actors."

The sql.js vs better-sqlite3 choice. The CX-I documented this as a binding guide fallback. The CX-C would document it as a configuration parameter: "The receipt store requires a SQLite engine with trigger support. The specific binding is a deployment configuration, not an architectural choice. Any SQLite binding satisfying INV-6 is admissible."

The vocabulary classification. The misclassification of `app_launch` as NATIVE_OPS instead of BOUNDARY_OPS caused 49 test failures. The CX-I fix was a cross-reference table. The CX-C would explain *why* app_launch is a boundary operation: "Process launch crosses the boundary between MosaicOS and the external operating system. It is governed at level B because MosaicOS mediates the boundary but does not own the external process."

In each case, the CX-C explanation provides context that the CX-I fix lacks. An agent reading only the CX-I knows *what* to do. An agent reading the CX-C knows *why*.

---

7. What the CX-S Contains

The Specification layer is the most abstract and the most stable. It changes only when fundamental understanding of the domain changes.

For a governance system, the CX-S would contain:

Domain invariants. Properties that every valid governance system must have. "Every mutation has a traceable cause" is a domain invariant. "Causes are

recorded as append-only receipts" is not – that's a configuration choice.

System taxonomy. What kinds of governance systems exist. A CX-S for computational governance might distinguish between advisory systems (governance produces recommendations), enforcement systems (governance prevents violations), and audit systems (governance records actions for later review). Each type has different invariants.

Configuration dimensions. The parameters along which valid configurations vary. For governance: traceability mechanism (receipts, logs, audit records), enforcement timing (pre-execution, post-execution, concurrent), authority model (hierarchical, delegated, capability-based), scope declaration (explicit, implicit, discovered).

Admissibility boundaries. Which combinations of configuration parameters are inadmissible. "Pre-execution enforcement with implicit scope" may be inadmissible because you can't enforce boundaries you haven't declared.

Theoretical foundations. References to the formal frameworks that underpin the domain. For Constitutional Computing: the relationship between governance and computation, the meaning of "constitutional" in the context of software systems, the epistemological status of receipts as evidence.

The CX-S educates agents about the domain. An agent that has read the CX-S understands not just the system it is building, but the class of systems to which it belongs.

---

8. What the CX-C Contains

The Configuration layer documents the specific architecture selected from the CX-S possibility space.

For the Constitutional Computing system, the CX-C would contain:

Configuration selections with rationale. "Traceability: append-only receipts. Rationale: independent verifiability without trusted third party. Alternative considered: structured logs. Rejected because: mutable, requires trust in log infrastructure."

Configuration topology map. A description (not necessarily exhaustive enumeration) of the configuration space neighborhood. "Adjacent configurations include: versioned receipts (relaxes append-only, gains amendment capability, loses simplicity), capability-based authority (replaces hierarchical delegation, gains composability, loses auditability of delegation chains)."

Blast radius declarations. For each configuration parameter: "If this parameter changed, the following CX-I phases would need modification." This enables impact analysis before reconfiguration.

Reconfiguration triggers. "If the following conditions are observed, this configuration should be reconsidered: (1) receipt store exceeds 10M entries and query performance degrades – consider indexed receipts with materialized views. (2) Multi-machine deployment required – consider replacing in-process authority with distributed capability tokens."

Design decision log. Every significant design decision, with date, alternatives considered, and the Governor's rationale. This is the institutional memory that currently lives in conversation transcripts and session histories.

---

9. The Naming Stack

The Constitutional Codex integrates with the broader system naming:

CX:AES – Constitutional Codex: Agent-Executable Specification. The complete three-layer methodology.

CX-S – Constitutional Specification. The domain possibility space.

CX-C – Constitutional Configuration. The selected design with rationale.

CX-I – Constitutional Implementation. The phase-gated build plan (formerly the Implementation Codex).

Meta-CX:AES – The meta-specification governing the structure of all Constitutional Codices.

UGA – Universal Governance Architecture. The theoretical framework.

UGK – Universal Governance Kernel. The runtime library implementing UGA.

BUGA – Bootstrapped UGA. Any system built on UGK.

A BUGA system has a CX. The CX has three layers. Agents build from the CX-I, consult the CX-C for design rationale, and reference the CX-S for domain understanding. Governors author and refine all three layers through observation-driven corrigibility.

---

10. Distinguishing from Existing Frameworks

10.1 Versus Formal Methods

Formal specification languages (TLA+, Alloy) operate at the CX-S level – they define what is possible and prove properties about configurations. But they do not produce implementation, and they do not capture configuration rationale. The Constitutional Codex spans all three SCI layers and produces constructible artifacts at each level.

10.2 Versus Architecture Decision Records

ADRs document configuration decisions with rationale – a partial CX-C. But ADRs are typically narrative documents without formal structure, they don't map the configuration topology, they don't declare blast radii, and they don't connect to implementation specifications. The CX-C formalizes what ADRs attempt informally.

10.3 Versus Domain-Driven Design

DDD's bounded contexts and ubiquitous language overlap with CX-S's domain analysis. But DDD is primarily a design methodology, not a specification framework. It produces design principles, not agent-executable artifacts.

10.4 Versus the Implementation Codex

The CX-I is the Implementation Codex. The Constitutional Codex adds context above it. The key advance is not replacing the CX-I but making explicit the layers of reasoning that the CX-I depends on but does not capture.

---

11. Limits

The Constitutional Codex does not claim that all domains have discoverable invariants, that all configuration topologies can be mapped, or that all design rationale can be formalized. It does claim that making these layers explicit – even partially – improves system understanding, enables more autonomous agent reasoning about design questions, and preserves institutional knowledge that would otherwise be lost in conversation transcripts.

The CX-S and CX-C are harder to validate empirically than the CX-I. The CX-I has a clear success criterion: cross-model convergence to zero observations. The CX-S and CX-C success criteria are less crisp – they are "complete" when agents can answer configuration-level questions without Governor intervention, but this is harder to measure than gate test pass rates.

The framework is most applicable to systems with well-defined domain structure – governance systems, protocol implementations, security architectures. Systems with emergent or discovery-driven requirements may benefit from the CX-I layer but may not have stable enough domain understanding for meaningful CX-S and CX-C layers.

---

12. Conclusion

The Constitutional Codex extends the Implementation Codex from a single-layer build plan to a three-layer specification framework that captures the full

reasoning stack from domain constraints through design decisions to construction procedures.

The defining property is corrigibility across all three layers. Observations at any layer propagate appropriately – implementation observations refine the build plan, configuration observations trigger design reconsideration, and specification observations update the domain model. The journal mechanism that proved effective for implementation refinement extends naturally to configuration and specification refinement.

The practical consequence is that a system specified under CX:AES carries its own explanation. An agent building from the CX-I knows *how*. An agent consulting the CX-C knows *why*. An agent reading the CX-S knows *what else could have been*. Together, the three layers constitute a complete, self-contained, corrigible specification – a constitutional codex from which a system can be realized.

Standing and Claim Boundary

This paper is a conceptual, formal, or architectural contribution unless it explicitly binds a statement to a cited external source or an exact carried evidence artifact. Unbound statements about implementation, validation, performance, safety, universality, or proof are research claims or design obligations, not established operational facts. Corpus-internal references establish conceptual lineage and comparison, not independent external confirmation.

Corpus-Wide Reference Positioning

This paper argues that the Implementation Codex (CX-I) demonstrated that software construction can be made deterministic through phase-gated, AI-executable specification. Its nearest external comparison fields are governance, formal, software. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E2**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E3**] Lawrence Lessig. *Code: Version 2.0.* Basic Books, 2006. https://codev2.cc/ **Relationship:** Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

[**E4**] OECD. *Recommendation of the Council on Artificial Intelligence.* OECD/LEGAL/0449, adopted 2019 and updated 2024. legalinstruments.oecd.org source **Relationship:** Adjacent policy standard for accountability, traceability, transparency, safety, and human-centered values.

[**E5**] George C. Necula. "Proof-Carrying Code." *POPL '97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E6**] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E7**] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL '77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

Ableman corpus

[**A1**] Adam Ableman. *Ableman Constitutional Governance*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-79353AF163A4B0F1`;
source SHA-256
`FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Theory*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D2D97C4042FD281F`;
source SHA-256
`FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Governance Engineering*. Ableman Corpus current edition; work identity `PAPER.PUB.18749870`;
source SHA-256
`372169F315D192733856B1F4A3F1038BBC73F57CF3ED9ACABAE72B965A69C8D0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Universal Governance Kernel (UGK)*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-1F9BFB2BF8370226`;
source SHA-256
`AA5106EA71E5DF120E141435C280DFC532715CA5DE0C75B8862BB7784ECCFACD`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Legitimacy Without Self-Ratification.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-EF2382F9397335AC`;
source SHA-256
`0186A39C808F8FDDB5C8F42384DF1A26678928434BD50A0EAE8D4947390F68A2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Ableman Corpus current edition; work identity `PAPER.PUB.21095303`;
source SHA-256
`EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Governed Semantic Compilation.* Ableman Corpus current edition; work identity `PAPER.PUB.18453681`;
source SHA-256
`5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Adam Ableman *Semantic Coordinate Identity Tokenization (SCIT).* Zenodo, 2026. doi:10.5281/zenodo.20532070

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848774